Guide operatoire
Parametres du workflow *Manual Auto Update KB* et demarrage rapide du runner self-hosted

Projet STM32Cube Preprocessing

20 août 2026

Table des matières

1 Objet du document **2**

2 Parametres du workflow *Manual Auto Update KB* **2**

3 Notions cles utilisees dans ce document **3**

4 Guide rapide : installation d'un runner self-hosted **3**

4.1 Prerequis machine 3
4.2 Creation du runner GitHub 3
4.3 Secrets obligatoires du depot 4
4.4 Preparation de l'environnement Python 4
4.5 Configuration Git sous Windows 4
4.6 Lancement du test superviseur 4
4.7 Resultat attendu 4
4.8 En cas d'echec 4

5 Document complementaire **5**

1 Objet du document

Ce document a deux objectifs :

— expliquer, champ par champ, les parametres du workflow GitHub Actions `Manual Auto Update KB` (`.github/workflows/manual_auto_update_kb.yml`), declenchable manuellement depuis l'onglet Actions ;

— fournir un guide rapide pour installer et configurer une machine de test equipee d'un runner self-hosted, prete a executer ce workflow.

2 Parametres du workflow *Manual Auto Update KB*

Le workflow est declenche via `workflow_dispatch` et propose les parametres suivants dans l'interface GitHub Actions.

Champ	Type / Defaut	Explication
`series`	texte, defaut vide	Codes de series STM32 separes par des virgules (ex : `H7,U5,WB`). Les valeurs autorisees sont derivees de `shared/config/config_all_series.json` (prefixe `STM32Cube` retire, ex : `STM32CubeH7` devient `H7`). Une valeur vide traite toutes les series.
`mode`	choix, defaut `Full`	Mode d'execution du script par serie. `Full` = pipeline complet (ingestion jusqu'a la livraison et l'upload). `Prepare` = genere uniquement les donnees ST-ready sans upload. `Upload` = upload seul, en supposant que les fichiers sont deja prepares.
`existing_datasource_mode`	choix, defaut `Replace`	Comportement si le datasource existe deja cote Alfred/KB. `Replace` = supprime et recree (rafraichissement complet). `Add` = ajoute en complement sans toucher a l'existant.
`skip_drivers`	booleen, defaut `false`	Si vrai, saute l'upload des drivers et sous-depots, pour ne traiter que les issues/fichiers/resolveurs.
`skip_schema_validation`	booleen, defaut `false`	Si vrai, desactive le garde-fou `validate_schemas.py` avant l'upload. Reserve au depannage, jamais en production.
`continue_on_workflow_error`	booleen, defaut `false`	Si vrai, lorsque `run_full_workflow` traite plusieurs depots et qu'un depot echoue, l'execution continue avec les depots suivants au lieu de tout arreter.
`placeholder_policy`	choix, defaut `Fail`	Politique appliquee si des placeholders Alfred (images/figures non resolues) subsistent dans les donnees. `Fail` bloque le pipeline, `Warn` journalise un avertissement et continue, `Off` ignore completement le controle.
`enable_issues_by_component_flow`	booleen, defaut `false`	Active la generation de datasources « issues » separes par composant, en plus ou a la place du datasource monolithique.
`component_issues_only`	booleen, defaut `false`	Si vrai, ne conserve que les datasources issues par composant et saute la version monolithique complete.
`max_parallel`	texte (nombre), defaut `'1'`	Nombre maximal de jobs de series executes en parallele dans la matrice GitHub Actions.

Champ	Type / Defaut	Explication
`upload_execution_target`	choix, defaut `self-hosted`	Runner utilise pour l'etape d'upload. `self-hosted` = runner local (acces reseau ST normal). `github-hosted` = runner cloud GitHub, reserve aux tests sans acces reseau interne ST.

3 Notions cles utilisees dans ce document

Cette section definit les termes techniques employes ci-dessus, afin d'eviter toute ambiguite de lecture.

— **Datasource (KB)** : unite de contenu enregistree dans la Knowledge Base Alfred (par exemple, les issues d'une serie, ou les fichiers d'un composant). Chaque datasource possede un identifiant numerique unique (`IssuesDatasourceId`, etc.) trace dans `shared/config/config_all_series.json`.

— **Alfred** : le service d'IA interne ST utilise pour enrichir le contenu (par exemple decrire des figures/PDF) et pour heberger la Knowledge Base consultee par le chatbot final.

— **Placeholder Alfred** : marqueur temporaire insere dans un document en attendant qu'Alfred fournisse une description (ex : pour une image ou un schema extrait d'un PDF). Un placeholder « non resolu » signifie que la description finale n'a pas encore ete generee ou integree.

— **Runner GitHub Actions** : machine qui execute les jobs d'un workflow. Un runner **self-hosted** est une machine geree par l'equipe projet (acces reseau interne ST), alors qu'un runner **github-hosted** est une machine ephemere fournie par GitHub dans le cloud, sans acces au reseau interne ST.

— **Job / matrice de jobs** : un *job* est une unite d'execution d'un workflow (ex : traiter une serie). Une *matrice* lance plusieurs jobs en parallele, un par valeur d'une liste (ex : une serie par job). Le parametre `max_parallel` limite combien de ces jobs tournent simultanement.

— **Schema de validation** : fichier de regles (via `validate_schemas.py`) qui verifie que chaque JSON de livraison ST-ready respecte la structure attendue (champs obligatoires, types) avant d'etre uploade vers la Knowledge Base.

4 Guide rapide : installation d'un runner self-hosted

Objectif : installer et configurer une machine de test en dix minutes.

4.1 Prerequis machine

— Windows 10/11.

— Git installe.

— Python 3.12 ou superieur installe.

— Machine sans veille (sleep/hibernate desactive).

— Connexion Internet stable.

4.2 Creation du runner GitHub

1. Ouvrir le depot GitHub concerne.
2. Aller dans *Settings* → *Actions* → *Runners* → *New self-hosted runner*.
3. Choisir *Windows x64*.
4. Executer les commandes proposees par GitHub sur la machine cible.
5. Installer le runner en tant que service et le demarrer.

4.3 Secrets obligatoires du depot

Aller dans *Settings* → *Secrets and variables* → *Actions* (onglet *Secrets*, pas *Variables*), puis ajouter :

— `RUNNER_CHECK_TOKEN` – verifie la disponibilite du runner self-hosted (job `runner-check`).

— `ST_GITHUB_ANALYZER_API_KEY` – cle prioritaire utilisee pour l'upload KB.

— `ST_CHATGPT_API_KEY` – cle ST ChatGPT, utilisee par le smoke test et en repli pour l'upload.

— `ST_REMOTE_USER` – utilisateur distant pour le precheck d'authentification upload (self-hosted).

Seuls des **secrets** sont necessaires ici ; aucune *variable* de depot n'est utilisee par ces workflows.

4.4 Preparation de l'environnement Python

Depuis la racine du depot :

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

4.5 Configuration Git sous Windows

```
git config --system core.longpaths true
```

4.6 Lancement du test superviseur

1. Ouvrir l'onglet GitHub Actions.
2. Lancer le workflow *Manual Auto Update KB*.
3. Choisir le mode `Full`.
4. Choisir une seule serie pour le test.

4.7 Resultat attendu

Ordre attendu des jobs :

1. Prepare matrix inputs
2. Check self-hosted runner availability
3. Preprocess <Serie>
4. Upload <Serie>

Controles a verifier :

— `runner_ready = true`

— Alfred API key preflight = OK

— Upload termine sans erreur

4.8 En cas d'echec

— **Erreur 403 sur le controle runner** : verifier `RUNNER_CHECK_TOKEN`.

— **Cle API Alfred manquante** : ajouter `ST_CHATGPT_API_KEY` (ou `ST_AI_BRIDGE_API_KEY` / `ST_API_KEY`).

— **Perte de communication avec le runner** : redemarrer le service runner, verifier le reseau ainsi que la RAM/CPU disponibles, et consulter les logs dans `actions-runner/_diag`.

5 Document complementaire

Ce guide est concu pour etre partage avec le guide *Rapport d'equipe – Ajout d'une nouvelle serie STM32Cube dans le pipeline* (`new_series_onboarding_and_runner_setup.pdf`), qui couvre l'integration complete d'une nouvelle serie dans le pipeline et dans les workflows CI/CD.